

Juiste examenvragen — Mobile IoT Apps

Dit document zet de voorbeeld-examenvragen netjes in Markdown en voegt per vraag een kort, examengericht antwoord toe.

1.1 Modern frontend build pipeline

1) Wat is het doel van een transpiler in een frontend build pipeline?

- A) Het combineren van meerdere bestanden tot één bundle
- B) Het omzetten van moderne code naar oudere compatibele versies
- C) Het uitvoeren van unit tests
- D) Het deployen van code naar productie
- E) Geen van bovenstaande is correct

Antwoord: B — moderne JS/TS omzetten naar een versie die oudere browsers/sneller tooling aankan.

2) Welke bundler is het meest geschikt voor snelle builds met minimale configuratie?

- A) Webpack
- B) Rollup
- C) Parcel
- D) Esbuild
- E) Geen van bovenstaande is correct

Antwoord: D — **esbuild** is extreem snel en vraagt weinig configuratie.

3) Wat gebeurt er met **logo.png** in de output bundle?

Gegeven:

```
npm install --save-dev esbuild
```

```
// package.json
{
  "scripts": {
    "build": "esbuild src/index.ts --bundle --outfile=dist/bundle.js --minify --sourcemap --loader:.png=dataurl"
  }
}
```

```
// src/index.ts
import logo from "./logo.png";
```

```
const img = document.createElement("img");
img.src = logo;
document.body.appendChild(img);
```

Antwoord: de PNG wordt **geïnlined als een data: URL** (base64/URL-encoded) in `dist/bundle.js` door `--loader:.png=dataurl`.

4) Waarom zou je Babel toevoegen bovenop TypeScript in deze pipeline?

Gegeven:

```
npm install --save-dev esbuild @babel/core @babel/preset-env esbuild-plugin-babel
typescript
```

```
// build.js
import { build } from "esbuild";
import babel from "esbuild-plugin-babel";

build({
  entryPoints: ["src/index.ts"],
  bundle: true,
  outdir: "dist",
  loader: { ".png": "dataurl" },
  plugins: [babel({ config: { presets: ["@babel/preset-env"] } })],
  platform: "browser",
  target: ["es2017"]
});
```

Antwoord: TypeScript verwijdert vooral **types** en doet beperkte syntax-transforms; Babel (met `preset-env`) kan **fijnmaziger transpilen** voor specifieke browsers/targets en (optioneel, met tooling) helpen met polyfills.

5) Vervolledig de GitHub Actions workflow voor CI met Vite + TypeScript (typecheck + build)

Antwoord (voorbeeld):

```
# .github/workflows/ci.yml
name: Vite CI

on:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
```

```
uses: actions/checkout@v4

- name: Setup Node
  uses: actions/setup-node@v4
  with:
    node-version: "18"
    cache: "npm"

- name: Install dependencies
  run: npm ci

- name: Type check
  run: npm run typecheck

- name: Build
  run: npm run build
```

1.2 React

1) Wat is het doel van de Virtual DOM in React?

- A) Het versnellen van netwerkverkeer
- B) Het direct aanpassen van de echte DOM
- C) Het efficiënt bijwerken van de UI door wijzigingen eerst in geheugen te verwerken
- D) Het opslaan van component state
- E) Geen van bovenstaande is correct

Antwoord: C — React berekent eerst in-memory verschillen en past daarna efficiënt de echte DOM aan.

2) Welke hook gebruik je om een component een interne toestand te geven?

- A) useEffect()
- B) useState()
- C) useRef()
- D) useContext()
- E) Geen van bovenstaande is correct

Antwoord: B.

3) Wat gebeurt er als je `setInterval()` direct in een component plaatst zonder `useEffect()`?

- A) De timer wordt één keer uitgevoerd
- B) De timer wordt verwijderd bij unmount
- C) Er ontstaan meerdere timers bij elke render
- D) React optimaliseert dit automatisch
- E) Geen van bovenstaande is correct

Antwoord: C — bij elke render maak je opnieuw een interval aan (memory leak/duplicatie).

4) Vul aan: werkende counter component met `useState`**Antwoord:**

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Counter: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

5) Gebruik props om een initiële waarde mee te geven aan de counter

Antwoord:

```
import React, { useState } from "react";

function Counter(props) {
  const init = parseInt(props.init || "0");
  const [count, setCount] = useState(init);

  return <h2>Startwaarde: {count}</h2>;
}

// Gebruik in App component:
// <Counter init={5} />
export default Counter;
```

6) Input component die toont wat je typt

Antwoord:

```
import React, { useState } from "react";

function InputEcho() {
  const [text, setText] = useState("");

  return (
    <div>
      <input
```

```
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>Je typte: {text}</p>
    </div>
  );
}

export default InputEcho;
```

1.3 Angular

1) Wat is het doel van de `@NgModule` decorator in Angular?

- A) Het definieert een nieuwe component
- B) Het maakt een service beschikbaar via dependency injection
- C) Het configureert een Angular module met componenten, imports en bootstrap
- D) Het koppelt een route aan een component
- E) Geen van bovenstaande is correct

Antwoord: C.

2) Vul aan: Angular component met `@Input()` om een naam te tonen

Antwoord:

```
import { Component, Input } from "@angular/core";

@Component({
  selector: "app-hero-name",
  template: `<h2>Hero: {{ name }}</h2>`
})
export class HeroNameComponent {
  @Input() name: string;
}
```

Gebruik in template:

```
<app-hero-name [name]='Batman'></app-hero-name>
```

3) Welke van onderstaande Angular directives is een structural directive?

- A) `ngClass`
- B) `ngStyle`
- C) `ngIf`
- D) `ngModel`

- E) Geen van bovenstaande is correct

Antwoord: C.

4) Gebruik `ngFor` om een lijst van dinosaurussen te tonen met hun index

Antwoord:

```
<ul>
  <li *ngFor="let dino of dinosaurs; let i = index">
    {{ i + 1 }}. {{ dino }}
  </li>
</ul>
```

5) Verschil tussen template-driven en reactive forms in Angular

- A) Template-driven forms zijn geschikt voor complexe validatie en unit testing
- B) Reactive forms zijn eenvoudiger te gebruiken voor beginners
- C) Template-driven forms gebruiken HTML als basis, reactive forms gebruiken TypeScript als basis
- D) Beide zijn identiek in werking
- E) Geen van bovenstaande is correct

Antwoord: C — template-driven: vooral in template (`ngModel`); reactive: `FormControl/FormGroup` in TypeScript (beter voor schaalbaarheid/testbaarheid).

6) Gebruik `ngModel` voor two-way databinding in een formulier

Antwoord:

```
<mat-form-field appearance="outline">
  <mat-label>Superhero Name</mat-label>
  <input
    type="text"
    [(ngModel)]="superhero.name"
    matInput
    placeholder="Enter name"
  />
</mat-form-field>
```

1.4 Angular met backend

1) Welke header moet aanwezig zijn om een Angular app toegang te geven tot een ASP.NET Web API op een ander domein?

- A) Authorization
- B) Content-Type
- C) Access-Control-Allow-Origin

- D) X-Requested-With
- E) Geen van bovenstaande is correct

Antwoord: C — dit is de kern van CORS: welke origins mogen de API aanspreken.

2) Wat is het verschil tussen een Web API met Entity Framework en een Web API met MongoDB?

Antwoord (kort):

- **Entity Framework (EF):** ORM voor (meestal) relationele SQL-databases, tabellen/relaties, migrations, LINQ.
- **MongoDB:** document database (JSON/BSON), flexibel schema, embedded documenten, andere query/index aanpak.

3) Vul aan: Angular service om data op te halen van een Web API

Antwoord:

```
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";

@Injectable({
  providedIn: "root"
})
export class BookService {
  private apiUrl = "http://localhost:5008/api/books";

  constructor(private http: HttpClient) {}

  getBooks(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }
}
```

4) Leg uit hoe SignalR verschilt van een klassieke HTTP-request in Angular. Wanneer verkies je SignalR?

Antwoord:

- **HTTP:** request/response (client vraagt, server antwoordt), niet real-time tenzij je polling gebruikt.
- **SignalR:** (meestal) persistente verbinding (WebSockets/fallbacks) waardoor de server **push**-berichten kan sturen.
- **Gebruik SignalR voor:** chat, live dashboards, notificaties, multiplayer/real-time updates.

5) Welke Angular package moet je installeren om met SignalR te werken?

- A) @angular/forms
- B) @angular/router

- C) @microsoft/signalr
- D) rxjs/websocket
- E) Geen van bovenstaande is correct

Antwoord: C.

1.5 Angular statemanagement met NgRx

1) Wat is het doel van een reducer in NgRx?

- A) Het ophalen van data uit een API
- B) Het transformeren van observables
- C) Het berekenen van nieuwe state op basis van een action
- D) Het dispatchen van een action naar de store
- E) Geen van bovenstaande is correct

Antwoord: C.

2) Welke hoort niet thuis in de NgRx Store volgens het SHARI-principe?

- A) State die gedeeld wordt tussen componenten
- B) State die wordt opgehaald via een side-effect
- C) State die niet serialiseerbaar is
- D) State die beschikbaar moet zijn bij route-herbetreding
- E) Geen van bovenstaande is correct

Antwoord: C — store state moet serialiseerbaar blijven (debugging, time-travel, persistence).

3) Wat doet de **debounceTime** operator in RxJS?

- A) Voert een API-call uit
- B) Wacht een bepaalde tijd na de laatste event voordat een waarde wordt doorgestuurd
- C) Verwijdt dubbele waarden uit een stream
- D) Combineert meerdere observables
- E) Geen van bovenstaande is correct

Antwoord: B.

4) Verschil tussen een smart component en een dumb component (NgRx)

Antwoord (kort):

- **Smart/container:** praat met store (select/dispatch), bevat "logica", haalt data op.
- **Dumb/presentational:** krijgt data via **@Input**, stuurt events via **@Output**, vooral UI.

5) Waarom is het belangrijk dat reducers pure functies zijn?

Antwoord: zodat dezelfde input (state + action) altijd dezelfde output geeft; dit maakt state voorspelbaar, testbaar en debugbaar (bv. time-travel).

6) Vul reducer aan: gebruiker toevoegen aan de state

Antwoord (typisch patroon):

```
function reducer(state: State = INITIAL_STATE, action: Actions): State {
  switch (action.type) {
    case "ADD_USER":
      return {
        ...state,
        users: [...state.users, action.user]
      };
    default:
      return state;
  }
}
```

7) Leg uit wat deze RxJS-code doet en waarom **pipe** en **mergeMap** belangrijk zijn

```
fromEvent(document, "click")
  .pipe(mergeMap(() => fetchDataFromApi()))
  .subscribe((data) => {
    console.log("Data ontvangen:", data);
  });
```

Antwoord: elke klik start een API-call; **pipe(...)** ketent operators; **mergeMap** "flattened" de inner observable/promise en laat meerdere calls **parallel** lopen (geen cancel).

8) Welke operator annuleert de vorige API-call bij snelle input?

Antwoord: **switchMap**.

```
fromEvent(inputElement, "keyup")
  .pipe(
    debounceTime(300),
    switchMap((event) => this.apiService.search((event.target as
HTMLInputElement).value))
  )
  .subscribe((result) => {
    console.log("Zoekresultaat:", result);
  });
```

9) Verschil tussen **mergeMap** en **switchMap** in RxJS

- A) mergeMap annuleert vorige observables, switchMap laat ze allemaal lopen
- B) switchMap annuleert vorige observables bij een nieuwe waarde, mergeMap laat ze allemaal lopen
- C) Beide operators combineren observables en wachten op voltooiing
- D) mergeMap is alleen geschikt voor HTTP-verzoeken, switchMap voor UI-events
- E) Geen van bovenstaande is correct

Antwoord: B.